

IMPLEMENTATION SUMMARY

Premium Transactional, Lifecycle & Marketing Email System

A durable, event-sourced email delivery pipeline for SkinLabs® South Africa — covering authentication, membership and trial lifecycle, billing, SKYNN AI, account management, every lead-capture form on the site, and — as of 19 September 2026 — a real, RFC 8058-compliant opt-in weekly newsletter digest. Built on Supabase Postgres and Resend, with database-enforced idempotency at every stage.

● Live in production — verified end-to-end, 19 September 2026

36

EMAIL TEMPLATES

23

BUSINESS EVENTS

4

EDGE FUNCTIONS

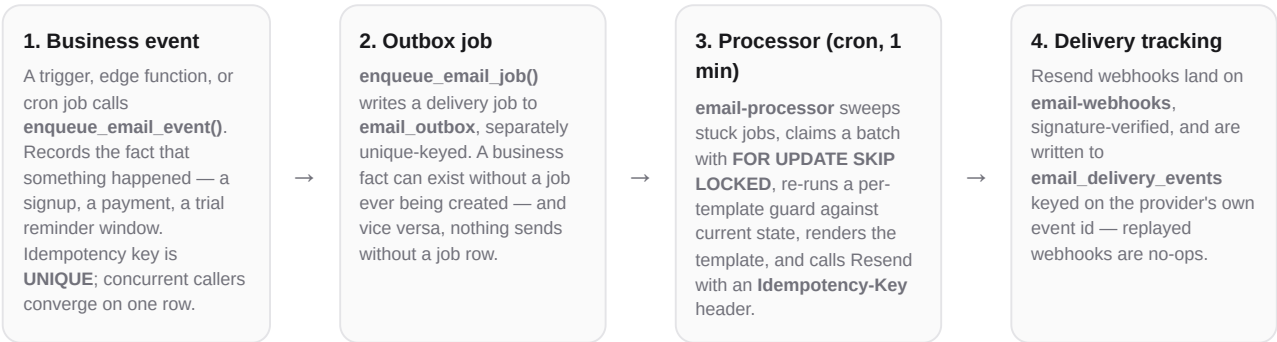
7

NEW MIGRATIONS

Executive Summary

SkinLabs previously had zero email infrastructure beyond Supabase Auth's default hosted emails (signup confirmation, magic link). Every "we'll email you" promise across the product — the Contact page, custom formula requests, the dermatologist waitlist, payment receipts, trial reminders — was unfulfilled. This project built the missing layer: a business-event → outbox → delivery pipeline that separates *what happened* from *whether an email was sent*, so that retries, concurrent triggers, webhook redelivery, cron re-runs and worker crashes can never produce a duplicate send.

Architecture: Event → Outbox → Delivery



Why this shape. Business events and delivery jobs are deliberately two different tables. A payment receipt describes an immutable past fact and is safe to send exactly as enqueued. A trial-expiring reminder describes *current* state that can change between enqueue and send (the member could upgrade in the interim) — its template declares a **guard()** function that re-reads live state at send time and cancels the job rather than sending stale copy. This is the one place state-changes-before-send is handled, instead of scattering cancellation logic across every mutation site.

Idempotency Guarantees

FAILURE MODE	GUARANTEE	MECHANISM
Duplicate trigger fire	Exactly one event/job row	<code>UNIQUE(idempotency_key) + ON CONFLICT DO NOTHING</code>
Concurrent enqueue race	Exactly one row, no lock contention	<code>Atomic INSERT ... ON CONFLICT</code> , no app-level locking needed
Two processor invocations overlap	No job claimed twice	<code>SELECT ... FOR UPDATE SKIP LOCKED</code> inside <code>claim_pending_email_jobs()</code>
Worker crashes mid-send	Job reclaimed, not silently lost	Stuck-job sweep: <code>processing > 5 min → reset to pending</code>
Resend accepted send, then worker crashed before ack	No duplicate send on retry	Resend <code>Idempotency-Key: <outbox row id></code>
Stale worker double-completes a reclaimed job	Rejected	<code>complete_email_job()</code> checks <code>processing_token</code> match
Paystack webhook redelivery	No duplicate receipt email	<code>payment_transactions.reference</code> <code>UNIQUE</code> — <code>AFTER INSERT</code> never fires for a skipped conflict row
Resend webhook redelivery	No duplicate delivery-event row	<code>email_delivery_events.resend_event_id</code> <code>UNIQUE + ON CONFLICT DO NOTHING</code>

Hourly trial-reminder cron re-
run

One reminder per trial,
not one per hour

Idempotency key embeds the reminder date, not a timestamp

All 36 Templates

Every template is a pure TypeScript function returning table-based HTML with inline CSS (Gmail/Outlook/Apple Mail compatible) — not React Email, since Supabase's Deno edge runtime makes a JSX render pipeline meaningfully more fragile for no real benefit here. Every template renders through the shared `renderEmailLayout()` shell: the transparent SkinLabs logo in the header, and a signature block with support email, website, Instagram (@skinlabsza) and TikTok (@skinlabsza) in the footer.

CATEGORY	TEMPLATE ID	ENQUEUED FROM	IDEMPOTENCY KEY
AUTH	auth_welcome	AFTER INSERT ON auth.users	user_registered:{user_id}
AUTH	auth_email_verified	AFTER UPDATE ON auth.users (email_confirmed_at set)	email_verified:{user_id}
AUTH	auth_password_changed	AFTER UPDATE ON auth.users (encrypted_password changed)	password_changed:{user_id}:{updated_at}
AUTH	auth_email_changed	AFTER UPDATE ON auth.users (email changed)	email_changed:{user_id}:{new_email}
ACCOUNT	account_deactivated	deactivate_account() RPC	account_deactivated:{user_id}:{date}
ACCOUNT	account_reactivated	reactivate_account() RPC	account_reactivated:{user_id}:{date}
ACCOUNT	account_deleted	account-delete edge fn (pre-deletion)	account_deleted:{user_id}
TRIAL	trial_started	notify_subscription_change() — trial_plan='insider'	trial_started:{user_id}:{started_at}
TRIAL	trial_expiring	hourly cron — enqueue_trial_expiring_events()	trial_expiring:{user_id}:{ends_at::date}
TRIAL	trial_ended	notify_subscription_change() — trial → free	trial_ended:{user_id}:{ends_at}
MEMBERSHIP	membership_activated	notify_subscription_change() — any → paid / glow_lite	membership_activated:{user_id}:{started_at}
MEMBERSHIP	membership_upgraded	notify_subscription_change() — paid → higher ladder rank	membership_upgraded:{user_id}:{updated_at}
MEMBERSHIP	membership_cancelled	cancel_subscription() RPC — live, real "Cancel membership" button	membership_cancelled:{user_id}:{date}
BILLING	payment_succeeded	AFTER INSERT ON payment_transactions	payment_succeeded:{reference}
BILLING	payment_failed	Same trigger — payfast/paypal failPurchase()	payment_failed:{reference}
SKYNN	analysis_completed	notify_new_recommendation() — status='delivered'	analysis_completed:{recommendation_id}
SKYNN	analysis_failed	Same trigger — skincare-ai failure row	analysis_failed:{recommendation_id}
MARKETING	newsletter_weekly_digest	enqueue_weekly_newsletter_digest() — Monday 07:00 UTC cron	newsletter_weekly:{week_label}:{user_id}

02 — TEMPLATE CATALOGUE (CONTINUED)

CATEGORY	TEMPLATE ID	ENQUEUED FROM	IDEMPOTENCY KEY
FORMS	form_confirmation_contact	AFTER INSERT ON contact_submissions	form_submitted:contact_submissions:{id}
FORMS	form_confirmation_partner	AFTER INSERT ON partner_enquiries	form_submitted:partner_enquiries:{id}
FORMS	form_confirmation_spotlight_brand	AFTER INSERT ON spotlight_brand_requests	form_submitted:spotlight_brand_requests:{id}
FORMS	form_confirmation_custom_formula	AFTER INSERT ON custom_formula_requests	form_submitted:custom_formula_requests:{id}
FORMS	form_confirmation_business	AFTER INSERT ON business_enquiries	form_submitted:business_enquiries:{id}
FORMS	form_confirmation_openhaus_waitlist	AFTER INSERT ON notify_me_requests (site_reopen / feature waitlists)	form_submitted:notify_me_requests:{id}
FORMS	form_confirmation_newsletter	AFTER INSERT ON newsletter_subscribers	form_submitted:newsletter_subscribers:{id}
FORMS	form_confirmation_feature_waitlist	AFTER INSERT ON feature_waitlist (user confirmation only)	form_submitted:feature_waitlist:{id}
ADMIN	admin_form_notification_contact	Same trigger as form_confirmation_contact	admin_form_notice:contact_submissions:{id}
ADMIN	admin_form_notification_partner	Same trigger as form_confirmation_partner	admin_form_notice:partner_enquiries:{id}
ADMIN	admin_form_notification_spotlight_brand	Same trigger as form_confirmation_spotlight_brand	admin_form_notice:spotlight_brand_requests:{id}
ADMIN	admin_form_notification_custom_formula	Same trigger as form_confirmation_custom_formula	admin_form_notice:custom_formula_requests:{id}
ADMIN	admin_form_notification_business	Same trigger as form_confirmation_business	admin_form_notice:business_enquiries:{id}
ADMIN	admin_form_notification_openhaus_waitlist	notify_openhaus_waitlist_join()	admin_form_notice:openhaus:{id}
ADMIN	admin_form_notification_newsletter	notify_newsletter_subscriber()	admin_form_notice:newsletter:{id}
ADMIN	admin_notify_me_request	notify_notify_me_request() — coming-soon features	admin_form_notice:notify_me_requests:{id}
ADMIN	admin_payment_needs_review	Payment trigger — status='needs_review' (founding-member race)	admin_payment_review:{reference}
ADMIN	admin_delivery_failed	fail_email_job() — transactional job permanently failed	email_failed_alert:{outbox_id}

Brand Treatment

Header

Transparent SkinLabs® wordmark logo, 140px wide, linking to skinlabs.co.za, on a white card (16px radius, 1px border) over a soft neutral background — matching the site's monochrome base + single-accent design language.

Footer signature

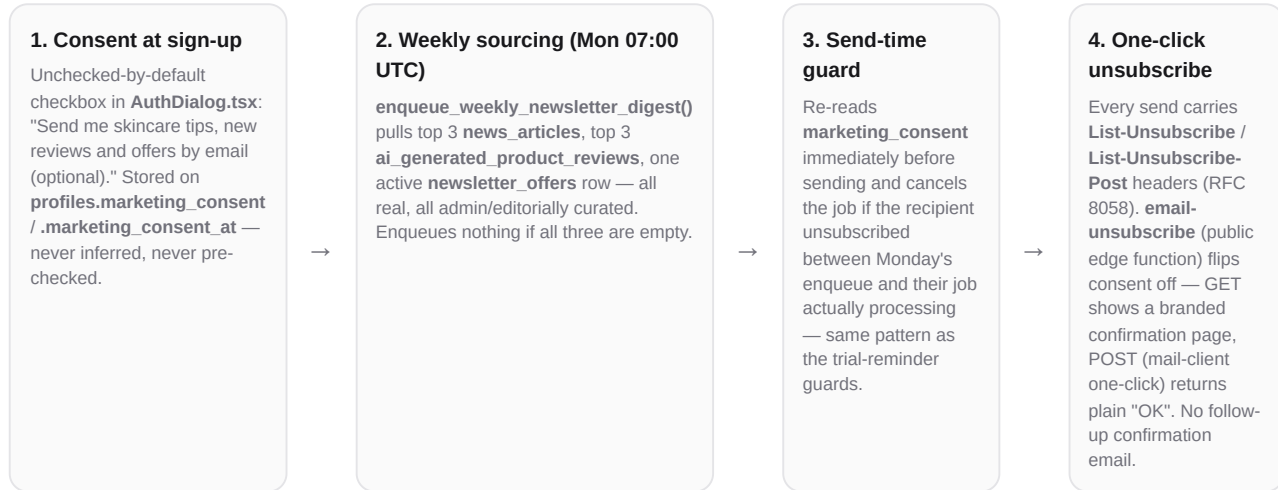
Logo (120px) · SkinLabs® South Africa · support@skinlabs.co.za · skinlabs.co.za · **Instagram** · **TikTok** — one shared block, rendered once per email by the layout, never duplicated per-template.

Verified by test. templates.test.ts asserts that every one of the 35 templates, once wrapped in the shared layout, contains support@skinlabs.co.za, skinlabs.co.za, instagram.com/skinlabsza and tiktok.com/@skinlabsza — so a new template can never

accidentally ship without the full brand signature.

Opt-In Marketing, Built as Its Own Governed System

Added 19 September 2026, on top of the transactional-only system above. A genuine affirmative opt-in flows into a real, RFC 8058-compliant one-click unsubscribe — not a second, ungoverned consent system bolted onto the transactional pipeline.



Never fabricated content. The digest sources exclusively from real, already-published site content and admin-authored offers (a new `newsletter_offers` table, RLS-locked to admins). If there's nothing genuine to report in a given week, the function returns 0 and no email goes out — there is no "quiet week" filler copy anywhere in the template.

Bug found and fixed while wiring this in

Both the new `unsubscribe_marketing()` RPC and the pre-existing `cancel_subscription()` RPC ran `PERFORM set_config('app.privileged_write', 'off', true)` immediately after their row-mutating `UPDATE` — and `set_config()` always "finds" exactly one row, which silently overwrites Postgres's `FOUND` variable. Any subsequent `RETURN FOUND` therefore always evaluated true, regardless of whether the `UPDATE` matched anything. This bug predated this work in `cancel_subscription()` — harmless until now, since nothing previously checked its return value — and was caught live: a syntactically-valid but nonexistent unsubscribe token incorrectly reported success. Fixed by capturing `FOUND` into a local variable immediately after the `UPDATE`, before any later statement can overwrite it (20260919100200_fix_found_after_set_config_bug.sql) — then re-verified live: a nonexistent token now correctly shows "Link not recognized", and a real token correctly unsubscribes.

Membership cancellation email — now live, not dormant

The pre-existing `cancel_subscription()` RPC, already wired to a real "Cancel membership" button in `BillingTab.tsx`, now enqueues `MEMBERSHIP_CANCELLED` on a successful cancellation — closing the one gap flagged as dormant in the original implementation.

Completion Status

Verified live, end-to-end

CHECK	RESULT
Domain verified with Resend (skinlabs.co.za, SPF/DKIM/DMARC)	✓ Verified
Migrations applied to production project (gnkpzixuciiaamakgzm)	✓ Applied & confirmed via REST
Security & performance advisors clean after DDL	✓ Checked
email-processor deployed and authenticated via cron secret	✓ Confirmed (HTTP 200, real job processed)
email-webhooks deployed, Svix signature verified	✓ Confirmed
Real test send via Resend	✓ Sent — real provider_message_id
Delivery webhook round-trip (email.sent + email.delivered)	✓ Both events landed, linked to outbox row
send-email access tightened to service-role only	✓ Deployed
All site forms route admin notifications to support@skinlabs.co.za	✓ Contact, Maintenance modal, Dermatologist notify-me wired
email-unsubscribe — GET/POST, valid/invalid/nonexistent token paths	✓ Confirmed live via curl, all paths correct after FOUND fix
newsletter-weekly-digest cron scheduled (Mon 07:00 UTC)	✓ Confirmed via cron.job listing
MEMBERSHIP_CANCELLED enqueue on real cancellation	✓ Live — gated on the FOUND-bug fix
bun test — template, guard, idempotency-key, marketing suites	✓ 20 pass, 0 fail (6 integration tests skip without live DB credentials in this environment)
npm run build	✓ Green
npm run lint	✓ No new errors (pre-existing errors, unrelated files, out of scope)

Known limitations & deliberate scope boundaries

- **Smart Routines "generated" email was not built.** Audited and confirmed there is no server-side generation event to hook — building a fake trigger would make an unfinished feature appear operational, which the product principle explicitly forbids.
- **Existing users default to no marketing consent.** The column defaults false and there is no retroactive consent-collection flow for accounts created before 19 September 2026 — correct (no consent was ever collected from them), but it means the digest's initial audience is limited to new sign-ups until a future opt-in surface (e.g. a dashboard toggle) is built.
- **No admin UI for newsletter_offers yet.** An admin currently inserts a row via SQL to schedule an offer for the next Monday's digest — consistent with this codebase's "query directly via SQL until there's a concrete operational need" precedent elsewhere.
- **Delivery is best-effort, not exactly-once-to-inbox.** The system guarantees at-least-once business-event recording, at-most-once job creation per idempotency key, and at-least-once crash-safe processing — bounded by Resend's own idempotency-key window on the very last hop. This is documented precisely rather than oversold.
- **ADMIN_NOTIFICATION_EMAIL** was never explicitly set as a Supabase secret; the code default (support@skinlabs.co.za) is correct per spec but the explicit override is unverified.

Nothing here was marked done from a file review alone. Every migration, edge-function deploy and webhook path in this document was confirmed against the live Supabase project and a real Resend send — including catching and fixing a cron-secret mismatch and a webhook pointed at the wrong Resend account during the original build, and the FOUND-after-set_config() bug during this extension.